Endpoint Investigator — Documento Técnico

Disciplina: Tecnologias Hackers — Avaliação Intermediária
Dupla: Kauã Figueira Santos · Matheus Constante
Professor: Dr. Rodolfo Avelino — Insper, Engenharia de Computação
Ferramenta: Endpoint Investigator v1.0.0 · schema `ei-evidence-bundle/1.0`

---

1. Problema

Uma evidência isolada quase nunca caracteriza um incidente. Um serviço que roda como root não é uma vulnerabilidade; um arquivo `0777` não é uma exploração; um processo chamado `curl` não é um canal de comando e controle. O que transforma observações soltas em algo investigável é a **relação** entre elas — e é exatamente isso que ferramentas que apenas reproduzem a saída de `ps`, `ls -l` e `systemctl` não entregam.

O problema que atacamos não é "listar informações do endpoint", e sim **reconstruir as relações entre identidade, processo, recurso e serviço**, e apresentar o resultado de modo que o analista separe o que foi observado do que foi inferido.

2. Estratégia de investigação

A ferramenta responde a uma pergunta central: *existe alguma fronteira de identidade atravessada neste host?* Ou seja, existe recurso que uma identidade executa e que **outra** identidade pode alterar? É o raciocínio do próprio enunciado: serviço executa como root → utiliza script → script pode ser alterado por usuário não privilegiado → possível relação de privilégio insegura.

Três decisões orientam o desenho:

(a) Correlação antes de julgamento. Nenhuma regra classifica um processo pelo nome. Toda regra parte de uma relação observada entre no mínimo duas dimensões, e o achado apresenta o contexto que o gerou.

(b) Contexto antes de volume. A análise de permissões não varre o filesystem. Parte dos recursos que processos e serviços efetivamente usam e se expande apenas para o entorno imediato: diretório-pai (que permite substituição), vizinhos do mesmo diretório (mesma zona de confiança), uma linha de base curta de arquivos sensíveis e o alvo real de cada link simbólico.

(c) Honestidade sobre o que não se sabe. Todo achado carrega o que ainda falta para confirmá-lo ou rejeitá-lo, e em que condições seria um falso positivo.

3. Arquitetura

```
COLETA → NORMALIZAÇÃO → CORRELAÇÃO → EVIDÊNCIAS → HIPÓTESES → RESULTADO
```

Camada	Módulo	Responsabilidade
1 Coleta	`collectors/`	Dois coletores intercambiáveis (live-linux e dataset) produzem o mesmo `Snapshot`. Trocar a fonte não altera nenhuma camada posterior
2 Normalização	`normalize.py`	Cria as chaves que tornam a correlação possível: modo → flags, cmdline → argv/interpretador/script, linha de log → registro, endereço → alcance
3 Correlação	`graph.py`	Grafo dirigido: processo→usuário, processo→pai, processo→arquivo, serviço→processo, serviço→arquivo, arquivo→diretório, log→pid
4 Evidências	`correlation/`	15 regras consultam o **grafo**, não os arquivos de origem
5 Hipóteses	`scoring.py`, `timeline.py`	Confiança e risco determinísticos; linha do tempo unificada
6 Resultado	`reporting/`	Console, JSON e HTML a partir do mesmo bundle

Proveniência. Toda entidade coletada carrega um `SourceRef` (`coletor::artefato:localizador`), propagado a cada item de evidência. Qualquer linha do relatório pode ser reconferida na fonte: `dataset::permissions.csv:linha 3`, `live-linux::/proc/2417`.

Fontes. As três dimensões obrigatórias — processos, permissões e serviços — mais os logs do sistema, que entram porque a quarta correlação citada no enunciado é "processo + serviço + log → reconstrução temporal". O escopo para aí: conexões de rede, contas do sistema e arquivos abertos não são coletados, e isso está declarado em todo relatório.

4. Principais correlações

As quatro correlações do enunciado estão cobertas. Três ilustram a estratégia:

EI-R01 — serviço + permissão + usuário. O caso central. A unidade declara `ExecStart`; a normalização extrai dele o binário e o script; o grafo liga a unidade ao arquivo; a regra pergunta quem, além da identidade que executa, pode escrever nesse arquivo. Quando o conjunto de escritores é maior que o de executores, há fronteira atravessada. Severidade ALTA quando a identidade que executa é privilegiada, MÉDIA caso contrário — porque o risco é a travessia da fronteira, não o fato de ser root.

EI-R04 — permissão + ausência de vínculo com execução. A contrapartida deliberada de EI-R01. Um arquivo `0777` que nenhum processo e nenhuma unidade executa é relatado em BAIXA, dizendo explicitamente que falta o elo "identidade privilegiada executa o recurso" que o tornaria grave. Sem essa regra, a ferramenta ou silenciaria o problema ou o inflaria; com ela, nomeia a diferença entre configuração inadequada e relação de privilégio demonstrada.

EI-R14 — log + processo + usuário. Reconstrução temporal. A regra casa `Accepted <método> for <usuário> from <ip>` no journal com o PID do `sshd` registrado na linha, ancora esse PID no inventário de processos e percorre os descendentes por PPID. O resultado é "o que a sessão fez depois do login" — uma linha de base de atividade, não um achado de risco.

Cálculo de confiança. Confiança é função da corroboração, não de opinião: 1 dimensão → baixa, 2 → média, ≥3 → alta; uma fonte única rebaixa o resultado. O risco (0–100) é a severidade base ponderada por esse fator, com acréscimo quando existe cadeia de privilégio explicitamente reconstruída. Um achado grave sustentado por uma fonte não ultrapassa um moderado corroborado por três.

5. Decisões técnicas

Python 3.10+ sem dependências. Uma ferramenta de resposta a incidente precisa rodar em uma máquina que talvez não tenha rede nem `pip`.

Leitura de `/proc`, não de `ps`. O coletor lê `status`, `cmdline`, `exe`, `cwd` e `stat` diretamente, obtendo UID/GID numéricos, o executável resolvido e a detecção de binário apagado — informações que a saída de `ps` não dá.

O grafo é a interface das regras. Regras consultam `paths_between`, `processes_of_service`, `children_of`. Acrescentar uma fonte significa acrescentar nós e arestas; as regras existentes passam a enxergá-la sem alteração.

Casamento serviço↔processo conservador. Um serviço `/bin/bash /opt/x.sh` não pode capturar todo `/bin/bash` do host: quando a unidade invoca interpretador com script, apenas o script identifica o processo. O caso inverso — o kernel registra o interpretador na frente de um script com *shebang* (`python3 /usr/bin/networkd-dispatcher`) — é tratado por comparação de token exato. Existe um modo "frouxo" por nome de unidade, usado **apenas para evitar falso positivo**, nunca para sustentar um achado.

Links simbólicos. No Linux o modo de um symlink é sempre `0777` e o kernel o ignora; vale o do alvo. Tratar isso na normalização foi decisivo: na primeira execução ao vivo, **todos** os achados de severidade ALTA eram symlinks (`/sbin/init`, `/usr/bin/python3`). Após a correção, o mesmo host passou de 32 achados, 15 deles ALTA falsos, para 8 achados e nenhum ALTA.

Correções no gerador fornecido. Ao montar os datasets encontramos três defeitos no script da disciplina, cada um reproduzido antes de ser corrigido e hoje coberto por teste: `random_noise()` levantava `ValueError: too many values to unpack (expected 5)` ao reaproveitar a saída já construída de `scenario_normal()`, o que tornava o cenário aleatório **inalcançável**; `iso()` rotulava `-03:00` sobre um horário declarado em UTC, adiantando os CSV em três horas em relação ao `journal.log` e podendo falsear a correlação temporal da regra EI-R15; e `scenario_permission()` se identificava como `normal`. O arquivo recebido está preservado byte a byte em `tools/generate_dataset_original.py`; as mudanças afetam só a cópia do projeto e estão em `docs/correcoes-no-gerador.md`.

Faixas de documentação. O módulo `ipaddress` do Python classifica `198.51.100.0/24` e `203.0.113.0/24` (RFC 5737) como privadas. Os datasets da disciplina as usam para representar hosts externos, e herdar esse engano faria a ferramenta concluir errado sobre destinos.

6. Uso de Inteligência Artificial

A IA foi usada **no desenvolvimento** — apoio à programação, redação da documentação e revisão de texto —, o que o enunciado permite expressamente.

A ferramenta entregue não incorpora modelo de linguagem. Toda a análise é determinística: a coleta lê as fontes do sistema, a normalização padroniza, o grafo constrói as relações e 15 regras próprias produzem os achados. Não há chamada a LLM em nenhuma camada, não há dependência de rede e o mesmo snapshot produz sempre o mesmo conjunto de evidências.

Como a cláusula do enunciado sobre IA é condicional — *"caso a solução incorpore um modelo de linguagem"* —, ela não se aplica a esta entrega. O fluxo é `DADOS → REGRAS PRÓPRIAS → EVIDÊNCIAS ESTRUTURADAS`, sem etapa de modelo.

7. Limitações

Declaradas em todo relatório gerado: (1) snapshot pontual — o que ocorreu antes ou depois da coleta não é observado, e PIDs podem ser reaproveitados; (2) sem inspeção de conteúdo de arquivo nem análise de binário; (3) conexões de rede, contas do sistema e arquivos abertos não são coletados, então uma URL em linha de comando não prova que houve conexão; (4) ACLs POSIX estendidas, atributos, *capabilities* e MAC não são lidos; (5) sem root, `cmdline` e executável de processos de outros usuários ficam incompletos, e a coleta registra isso; (6) namespaces de containers não são percorridos; (7) a ferramenta **não conclui** que houve comprometimento — todo achado carrega `exploracao_comprovada: false`.

O princípio: quando a ferramenta não tem como distinguir duas explicações, ela apresenta as duas e diz qual evidência resolveria a dúvida. A lista completa de falsos positivos conhecidos, com o tratamento de cada um, está na seção 8 do README.

8. Validação

- **36 testes** sem dependências (`python3 -m unittest discover -s tests`), cobrindo normalização, proveniência de toda evidência, o casamento serviço↔processo, as quatro correlações do enunciado, a presença dos quatro elementos em todo achado e as correções feitas no gerador.
- **6 datasets** com cenário escolhido por nome e seed fixa: normal, permissão ampla sem execução, serviço privilegiado legítimo, correlação, ambiguidade e o cenário aleatório do gerador.
- **Cenário controlado** na própria máquina (`tools/lab_scenario.sh`), sem root e reversível, e dois testes de não-alarme — `test_cenario_normal_nao_gera_achado_alto` e `test_servico_root_sozinho_nao_e_vulnerabilidade` — garantindo que a ferramenta não inventa risco onde não há correlação.